

One Counter per Resource Family: Compact Online Multicore Phase Prediction

Anonymous Author(s)

Abstract—Hardware performance counters can expose workload phases, but a full counter vector often exceeds the physical PMU slots and a high-accuracy phase predictor can obtain a misleading result by merely repeating an oracle phase label. We study a stricter interface: an offline, train-only full-counter teacher defines a global pressure phase, while an online predictor observes one validation-selected counter state from each resource family. We separate oracle diagnostics from deployable baselines, split and resample complete executions, and include the discretizer and history state in the storage budget. In an audited pilot of 60 PARSEC executions, a depth-six tree over five 20-interval histories obtains 98.21% held-out accuracy (95% execution-group bootstrap CI: 97.64–98.71%) using an estimated 201.25 bytes. Against an equally deployable tree using only the current counter states, history improves accuracy by 0.60 percentage points (paired 95% CI: 0.29–0.92) and transition accuracy by 12.95 points, although the latter CI includes zero. The audit also finds material multiplexing variation in the pilot collection, showing why enabled-time and workload-disjoint results must accompany accuracy. These results motivate a measurement-quality-first evaluation of compact phase interfaces rather than an uncalibrated claim of end-to-end hardware savings.

Internal evidence-audit draft—do not submit. The final multicore files are not mounted in the current environment. Numbers marked “pilot” come from the older five-workload validation set and are included to test the analysis and paper structure, not to replace the required final rerun.

I. INTRODUCTION

An online hardware controller needs a compact description of what the workload will do next. Raw performance-monitoring-unit (PMU) events are natural signals, yet three problems make a phase interface difficult to evaluate. First, processors expose more useful events than can be scheduled simultaneously; multiplexing can produce noisy or missing 10 ms samples. Second, adjacent phase labels are highly persistent, so a model can report high accuracy by repeating the current phase while detecting no changes. Third, randomly splitting intervals leaks nearly identical samples from one execution into both training and test sets. These problems matter before the choice of model.

Prior work established code- and counter-based program phases for representative simulation, runtime prediction, cache adaptation, and multicore management [1], [2], [3], [4], [5], [6]. Rich offline signatures are valuable for phase discovery but are not automatically viable as a short-latency hardware interface. Conversely, a tiny model is not useful if it relies on an oracle current label or a counter set that the PMU rarely schedules.

We investigate whether a global phase discovered from a broad safe-counter vector can be approximated online by one counter per resource family. The families correspond to L1, L2,

LLC, branch/control, core/FP, and memory/off-core behavior. A train-only k -means teacher creates ordered pressure labels. Validation ablation freezes one event per available family. One-dimensional train-only clustering converts each event to a two-bit state, and a shallow tree predicts the next global label from the last H states. We call this pipeline FAMILYPHASE.

The paper makes three methodological contributions.

- We define a reduced resource-family PMU interface while keeping the target explicit: every student predicts one *global full-counter teacher label*; the per-family states are inputs, not separate phase ground truths.
- We introduce an evaluation protocol that groups co-running processes and repetitions before splitting, labels persistence/Markov models that consume the teacher’s current phase as oracle diagnostics, and compares history against a deployable current-state-only tree.
- We make collection quality and implementation cost first-class outputs: execution-group confidence intervals, k /seed stability, PMU enabled time, tree state, discretization thresholds, and history storage are reported together.

The audited pilot suggests that compact histories contain useful transition information, but it also exposes evidence that must be corrected before a full claim: only nine runs are held out in the primary split, $k = 2$ is more stable than the chosen $k = 3$, and several events have poor enabled time. We report these limitations rather than converting them into a proxy energy result.

II. BACKGROUND AND RELATED WORK

A. Program phases

BBV and SimPoint methods cluster intervals with similar basic-block distributions and use representative intervals to reduce simulation cost [1], [2], [7]. Online phase trackers replace long profiling signatures with compact histories and transition models [3]. The definition is task dependent: working sets are useful for cache allocation, while PMU events expose resource pressure with lower software instrumentation cost [8], [9].

Multicore execution complicates phase ownership. A global label represents a package or workload group cheaply, a local label represents each core independently, and a distributed predictor combines local and cross-core context [4], [10], [6]. FAMILYPHASE uses a global target because package-level cache, bandwidth, thermal, and scheduling policies need a common state. Resource-family inputs describe which subsystems provide evidence for that target; we do not call them family-specific target phases.

B. Counter selection and phase-aware control

Counter-based predictors have been applied to power/performance modeling, scheduling, and cache-demand estimation [11], [5], [12]. Phase-aware policies have also controlled caches, dormant structures, DVFS, and power balancing [13], [14], [15], [16], [17]. These works motivate the resource-family organization. Our focus is narrower: how much phase fidelity survives an aggressively reduced observable interface, and how should that trade-off be measured without leaking executions or granting a baseline oracle labels?

PMU event meaning and availability vary across microarchitectures. Multiplexed events are scaled by Linux `perf`, but enabled time still controls measurement quality and should be reported [18], [19], [20], [21]. This motivates separating teacher collection, reduced online collection, and model evaluation instead of counting only final model parameters.

III. RESOURCE-FAMILY PHASE INTERFACE

A. Train-only global teacher

Let $x_{r,t} \in \mathbb{R}^d$ be the safe PMU vector at interval t of run r . Timing-derived features such as cycles, IPC/CPI, elapsed time, and requested frequency are excluded so that a future policy does not trivially feed its action back into the label. Each nonnegative count is transformed by $\log(1+x)$, missing values are filled with training medians, and features are standardized with training statistics.

The teacher fits k -means on training rows only:

$$c_t = \arg \min_{j \in \{0, \dots, k-1\}} \|\tilde{x}_{r,t} - \mu_j\|_2^2. \quad (1)$$

Validation and test rows use the frozen scaler and nearest centroid. Anonymous cluster IDs are ordered by an increasing pressure score over available LLC-miss, memory- bandwidth, branch-misprediction, and stall indicators. For $k = 3$, the labels are reported as low, moderate, and high pressure. The labels are descriptive operating regions, not an external ground truth for power or bottlenecks.

B. Selecting and discretizing the online interface

Candidate events are partitioned into resource families. Near-duplicate events are identified on training data using Spearman correlation. Singleton decision-tree ablations are scored on validation accuracy and high-pressure recall, and one representative e_f^* is frozen per available family f . The test split never selects an event.

For the online representation, a one-dimensional three-cluster model is fit to e_f^* on training rows. Its ordered state $s_{f,t} \in \{0, 1, 2\}$ needs two bits. Two 16-bit boundaries per family approximate the nearest-centroid decision. The cross-family history vector is

$$u_t = [s_{1,t-H+1}, \dots, s_{F,t-H+1}, \dots, s_{F,t}], \quad (2)$$

and the deployable predictor learns $h(u_t) \rightarrow c_{t+1}$. A depth-limited CART tree is used because inference consists of comparisons and table-like child traversal [22], [23].

TABLE I
MODEL INFORMATION AVAILABLE AT PREDICTION TIME.

Model	Online input	Deploy.
Majority	training prior	yes
Teacher	persistence true teacher c_t	no
tence/Markov		(oracle)
Current-state tree	selected $s_{1..F,t}$	yes
Single-family history tree	selected $s_{f,t-H+1:t}$	yes
Cross-family history tree	selected $s_{1..F,t-H+1:t}$	yes

C. Baselines and their information sets

Table I prevents an easy but consequential category error. Persistence, state-conditioned majority, Markov, run-length Markov, and the duration approximation consume the teacher's true c_t . Because the full teacher is offline, these methods are oracle diagnostics unless a separate online phase estimator is added. They measure phase persistence but are not fair hardware implementations. The majority predictor uses only the training label prior. The fair history ablation is a tree trained on the current vector $[s_{1,t}, \dots, s_{F,t}]$ with the same depth/leaf limits.

D. Analytical implementation budget

For a tree with I internal nodes, L leaves, D input features, and $N = I + L$ nodes, the model estimate is

$$B_{tree} = \frac{I(\lceil \log_2 D \rceil + 16 + 2\lceil \log_2 N \rceil) + 2L}{8}. \quad (3)$$

The full online estimate adds $\lceil 2FH/8 \rceil$ history bytes and $4F$ bytes for two 16-bit discretizer boundaries per family. This excludes PMU register hardware already present in the processor and is an analytical storage estimate, not synthesis area.

IV. EXPERIMENTAL METHODOLOGY

A. Audited pilot

The currently reproducible pilot uses five PARSEC workloads—blackscholes, bodytrack, canneal, fluidanimate, and freqmine—with 1, 2, 4, and 8 threads and three repetitions per configuration (60 runs) [24]. Task-local counters were sampled at 10 ms on a dual-socket Intel Xeon E5-2680 v2 system with 20 physical cores and SMT disabled, using Linux `perf` 5.14. The safe merged table contains 9,409 interval rows and eight usable counter columns. L2 events were unavailable, leaving five online families. This dataset is a method-validation pilot, not the intended final multicore co-runner study.

The primary split groups all repetitions of each workload/thread configuration before assigning 70%/15%/15% to train/validation/test. There are 5,642 valid history windows for training and 1,341 for test; only nine complete runs occur in the test split. Leave-one-workload-out results are generated separately. All interval metrics are accompanied by a cluster bootstrap that samples complete run or concurrent-group IDs.

B. Final multicore protocol

The final study uses three regimes: one multithreaded process, two single-threaded co-runners, and two four-thread co-runners. For seven workloads the collector enumerates all $\binom{7}{2} = 21$ unordered cross-workload pairs. Both processes in a concurrent group share a split, and any workload holdout moves every group containing that workload to test. This corrects an earlier exploratory design that used only four fixed pairs and could split partners independently. No result from that old split is used in this draft.

C. Metrics and uncertainty

We report accuracy, macro F1, balanced accuracy, high-pressure recall, stable-case accuracy, and transition-case accuracy. A transition case satisfies $c_{t+1} \neq c_t$. Final results will additionally report transition precision/recall/F1, false alarms, and transition detection delay. Confidence intervals use 10,000 execution-group bootstrap resamples. Model comparisons use paired resampling of the same groups; a difference is not described as reliable when its 95% interval crosses zero.

Clustering robustness is evaluated for $k \in \{2, 3, 4, 5\}$ and five initializations using pairwise adjusted Rand index (ARI), train distance, centroid separation, and minimum cluster share. History, tree depth, and sampling-interval sensitivity are also required for the final study.

V. AUDITED PILOT RESULTS

A. Does history help a deployable predictor?

Table II reports one copy of each cross-family result. The exporter uses the family label `__all_families__` for these rows because the global target and cross-family features must not be repeated and averaged as independent observations. The oracle persistence diagnostic reaches 97.69% accuracy because only 31 of 1,341 test windows transition, but it has zero transition accuracy. Its high accuracy is evidence of label persistence, not a deployable win.

The current-state tree recovers 97.61% accuracy and correctly predicts 12.9% of transitions. The 20-interval history tree improves accuracy to 98.21% and transition accuracy to 25.8%. The execution-group CIs are wider than an interval bootstrap would suggest. A paired comparison finds a 0.60-point accuracy improvement (95% CI: 0.29–0.92; bootstrap probability of a positive difference 99.99%). The transition improvement is 12.95 points, but its 95% CI is 0.0–26.92 points. Thus this pilot supports the accuracy effect but is underpowered for a strong transition claim. As a binary change detector, the history tree has 90.0% precision, 29.0% recall, and 43.9% F1 (9 true changes and one false alarm); the current-state tree has 54.5% precision, 19.4% recall, and 28.6% F1 (6 true changes and five false alarms).

The majority transition accuracy is not a contradiction: with few transitions, a constant phase can happen to match one transition destination while failing most stable regions. This is why neither overall accuracy nor transition accuracy alone is sufficient. Macro F1 and the confusion matrix must accompany both.

TABLE II

PILOT CONFIGURATION-GROUP HOLDOUT. CIS RESAMPLE NINE TEST RUNS. ORACLE PERSISTENCE CONSUMES THE OFFLINE TEACHER PHASE AND IS NOT DEPLOYABLE.

Model	Accuracy (95% CI)	Macro F1	Trans. acc.
Majority	77.03 (70.77–82.69)	29.01	48.4
Oracle persistence	97.69 (97.20–98.14)	95.97	0.0
Current-state tree	97.61 (96.84–98.30)	95.45	12.9
20-history tree	98.21 (97.64–98.71)	96.88	25.8

TABLE III

PILOT LEAVE-ONE-WORKLOAD-OUT RESULTS (PERCENT).

Holdout	Cur. acc.	Hist. acc.	Cur. trans.	Hist. trans.
blackscholes	97.68	98.24	21.43	40.48
bodytrack	76.47	82.24	21.04	47.26
canneal	98.22	98.28	25.71	45.71
fluidanimate	95.77	96.69	20.27	50.00
fregmine	96.94	97.55	24.14	36.21
Mean	93.01	94.60	22.52	43.93

B. Does the interface generalize to an unseen workload?

Table III holds out every run of one workload at a time. Across the five folds, the history tree averages 94.60% accuracy and 43.9% transition accuracy, compared with 93.01% and 22.5% for the current-state tree. Oracle persistence averages 94.03% accuracy and zero transition accuracy. The result is not uniformly easy: bodytrack reduces history-tree accuracy to 82.24%. This failure case should be explained with its phase shares and confusion matrix rather than hidden by the mean. Paired execution-group bootstraps place the transition-accuracy improvement above zero in every fold (the smallest lower bound is 5.9 points). Accuracy improves reliably in four folds; canneal’s 0.07-point difference has a 95% CI of -0.38 – 0.63 points. Because folds reuse training workloads, their arithmetic mean is descriptive rather than five independent replications.

C. What does the compact interface cost?

The current-state tree uses an estimated 64.88 B including five discretizers and a two-byte current-state register. The cross-family history tree uses 201.25 B including 25 B of two-bit history and 20 B of thresholds. The increase is small in absolute terms, but it should not be advertised as RTL area without synthesis. Software inference latency in the Python implementation is approximately 52 μ s per prediction and is not a cycle-level hardware estimate.

D. Are three teacher states stable?

Table IV shows that the pilot does not uniquely justify $k = 3$. Three states have mean pairwise ARI 0.927, but the smallest cluster averages only 2.1% of rows. Two states are almost invariant to initialization and more balanced. Four and five states improve within-cluster distance but reduce seed stability and create even smaller clusters. A final paper may retain $k = 3$ for low/moderate/high semantics only if the larger multicore study confirms stability and utility; otherwise k should be selected on train/validation evidence.

TABLE IV
PILOT TRAIN-ONLY CLUSTERING SENSITIVITY OVER FIVE SEEDS.

k	Mean ARI	Min ARI	Train dist.	Min share
2	1.000	0.999	1.418	15.8%
3	0.927	0.817	1.276	2.1%
4	0.845	0.742	1.136	1.8%
5	0.734	0.511	0.990	1.8%

E. PMU scheduling is a first-order validity issue

Reprocessing exactly the 60 raw pilot runs preserves `perf`'s enabled percentage. For the eight safe events used by the teacher, mean enabled time is 90.6–91.0%, but the fifth percentile is only 64–66% and 32.4–33.7% of samples fall below 90%. Rare samples reach zero enabled time. This variation can distort both the teacher and the selected interface at 10 ms granularity. Accordingly, the final experiment must compare the full profile against the frozen reduced profile and report enabled-time improvement and runtime overhead. The pilot accuracy cannot establish that benefit by itself.

VI. DISCUSSION

A. What the result supports

The pilot supports a limited conclusion: within seen workload families and a grouped configuration split, five compact current states approximate the next global teacher phase, and a short history provides a small paired accuracy improvement. The analytical state is hundreds rather than tens of thousands of bytes. These properties make the interface plausible for a firmware or hardware prototype.

It does not establish energy savings, cache savings, or optimal control. The prior draft mapped labels to arbitrary L1 power factors; that proxy is removed because no cache size, miss latency, energy model, or slowdown was calibrated. A control claim requires Linux `resctrl`, a cache simulator, DVFS measurements, or another real actuator with performance and energy accounting.

B. Threats to validity

Construct validity. The teacher labels are clusters in the chosen counter space, not independently measured bottlenecks. Ordering them by pressure aids interpretation but does not prove that every high label needs the same action.

Internal validity. PMU multiplexing, frequency variation, OS noise, NUMA placement, affinity, warm-up, and interval alignment can change counts. The final artifact must report event enabled time, governor/turbo state, kernel and `perf` versions, exact event encodings, pinning, socket placement, unsuccessful samples, and run randomization.

Statistical validity. Intervals from one run are correlated. We therefore split and bootstrap runs/concurrent groups, but nine pilot test runs remain too few for a transition conclusion. Multiple seeds and paired group resampling reduce, but do not eliminate, uncertainty.

External validity. The pilot covers five PARSEC programs on one Ivy Bridge server with task-local events. Results may

not transfer to different PMUs, production services, SMT, other sockets, or unseen co-runners. Workload-disjoint, pair-disjoint, and preferably second-generation CPU experiments are required.

Implementation validity. Byte and comparison counts are analytical. They exclude counter read energy, control integration, timing closure, and actuator cost. RTL synthesis or carefully qualified firmware measurements are needed before calling the predictor implemented hardware.

VII. CONCLUSION

Compact phase prediction should be evaluated as a measurement interface, not only as a classifier. FAMILYPHASE separates an offline global teacher from one-counter-per-family online signals, distinguishes oracle persistence from deployable baselines, and reports execution-group uncertainty and complete interface storage. An audited pilot finds a small but reliable accuracy gain from history and a promising, not yet conclusive, transition gain. It also reveals material PMU enabled-time variation and k sensitivity. Those findings define the evidence required for the final multicore study: all co-runner pairs, reduced-profile overhead, workload-disjoint tests, robust phase selection, and no proxy energy claims.

REFERENCES

- [1] T. Sherwood, E. Perelman, and B. Calder, "Basic block distribution analysis to find periodic behavior and simulation points in applications," in *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*, 2001, pp. 3–14.
- [2] T. Sherwood, E. Perelman, G. Hamerly, and B. Calder, "Automatically characterizing large scale program behavior," in *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems*, 2002, pp. 45–57.
- [3] T. Sherwood, S. Sair, and B. Calder, "Phase tracking and prediction," in *Proceedings of the 30th International Symposium on Computer Architecture*, 2003, pp. 336–349.
- [4] E. Perelman, M. Polito, J.-Y. Bouguet, J. Sampson, B. Calder, and C. Dulong, "Detecting phases in parallel applications on shared memory architectures," in *Proceedings of the 20th IEEE International Parallel and Distributed Processing Symposium*, 2006.
- [5] J. Nomani and J. Szefer, "Predicting program phases and defending against side-channel attacks using hardware performance counters," in *Proceedings of the Fourth Workshop on Hardware and Architectural Support for Security and Privacy*, 2015.
- [6] E. S. A. Lozano and A. Gerstlauer, "Learning-based phase-aware multi-core cpu workload forecasting," *ACM Transactions on Design Automation of Electronic Systems*, vol. 28, no. 2, 2022.
- [7] G. Hamerly, E. Perelman, J. Lau, and B. Calder, "Simpoint 3.0: Faster and more flexible program phase analysis," *Journal of Instruction-Level Parallelism*, vol. 7, pp. 1–28, 2005.
- [8] A. S. Dhodapkar and J. E. Smith, "Managing multi-configuration hardware via dynamic working set analysis," in *Proceedings of the 29th International Symposium on Computer Architecture*, 2002, pp. 233–244.
- [9] —, "Comparing program phase detection techniques," in *Proceedings of the 36th Annual IEEE/ACM International Symposium on Microarchitecture*, 2003, pp. 217–227.
- [10] C.-H. Chang, P. Liu, and J.-J. Wu, "Sampling-based phase classification and prediction for multi-threaded program execution on multi-core architectures," in *Proceedings of the 42nd International Conference on Parallel Processing*, 2013, pp. 349–358.
- [11] Y. Kim, P. Mercati, A. More, E. Shriver, and T. Rosing, " p^4 : Phase-based power/performance prediction of heterogeneous systems via neural networks," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*, 2017, pp. 683–690.

- [12] I. E. Ziedan, H. I. Shehata, and S. M. Serag, "A run-time program phase detection technique for optimizing per-phase l2 cache demand," *The Egyptian International Journal of Engineering Sciences and Technology*, vol. 20, pp. 1–9, 2016.
- [13] R. Balasubramonian, D. H. Albonesi, A. Buyuktosunoglu, and S. Dwarkadas, "Memory hierarchy reconfiguration for energy and performance in general-purpose processor architectures," in *Proceedings of the 33rd Annual IEEE/ACM International Symposium on Microarchitecture*, 2000, pp. 245–257.
- [14] D. H. Albonesi, "Selective cache ways: On-demand cache resource allocation," in *Proceedings of the 32nd Annual IEEE/ACM International Symposium on Microarchitecture*, 1999, pp. 248–259.
- [15] A. Buyuktosunoglu, S. Schuster, D. Brooks, P. Bose, P. Cook, and D. Albonesi, "An adaptive issue queue for reduced power at high performance," in *Power-Aware Computer Systems*, 2001, pp. 25–39.
- [16] C. Isci and M. Martonosi, "Runtime power monitoring in high-end processors: Methodology and empirical data," in *Proceedings of the 36th Annual IEEE/ACM International Symposium on Microarchitecture*, 2003, pp. 93–104.
- [17] A. Merkel and F. Bellosa, "Balancing power consumption in multiprocessor systems," in *Proceedings of the 1st ACM SIGOPS/EuroSys European Conference on Computer Systems*, 2006, pp. 403–414.
- [18] Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual*, 2024.
- [19] —, *Intel 64 and IA-32 Architectures Software Developer's Manual*, 2026.
- [20] Linux Kernel Developers, *perf_event_open(2) Linux Programmer's Manual*, 2024.
- [21] Linux perf developers, "perf: Linux profiling with performance counters," <https://perfwiki.github.io/main/>, 2024.
- [22] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Wadsworth, 1984.
- [23] J. R. Quinlan, *C4.5: Programs for Machine Learning*. Morgan Kaufmann, 1993.
- [24] C. Bienia, S. Kumar, J. P. Singh, and K. Li, "The parsec benchmark suite: Characterization and architectural implications," in *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques*, 2008, pp. 72–81.